university of groningen / faculty of science and engineering

Nanoscale Material Modelling

Week 4

Ilija A. Gjerapić, S4437586; i.a.gjerapic@student.rug.nl; @github
Supervisor: prof. dr. A. Giuntoli, prof. dr. J. Slawinska

Contents

1 Assignment 1: The Icebreaker

The TIP3P and TIP4P models of water were used to model the melting of ice I_H by increasing the simulation temperature from 0K to 1000K. The main differences between these two models is the location of the charges. TIP3P has each charge localized at the center of each atom, while the TIP4P model localizes the charge of the Oxygen closer to the two hydrogen atoms in a forth massless particle. Table 1.1 shows the difference in efficiency between the two models.

Table 1.1: A comparison between the efficiency of the two water models. The introduction of the fourth, massless particle in the TIP4P water model results in an efficiency drop of around 55%, when considering timesteps/s.

Water Model	hours/ns	timesteps/s
TIP3P	0.225	1233.335
TIP4P	1.363	679.128

The radial distribution function (RDF) at increasing temperatures for the `TIP3P` model and `TIP4P` are shown in Figure 1.1(a) and 1.1(b), respectively. It is clear that at higher temperatures, the strucure for both models becomes more liquid like as the sharp peaks disipate. This is clearer when visualizing the initial and final structures of the two models as shown in the insets of Figure 1.2.

However, peaks at 1.0 Å and 1.5 Å are persistant no matter the temperature. This is as these peaks relate to single water molecules: the peak at 1.0 Å corresponds to the distance between the oxygen and hydrogen atoms, while the peak at 1.5 Å is due to the fixed distance of the two hydrogen atoms.

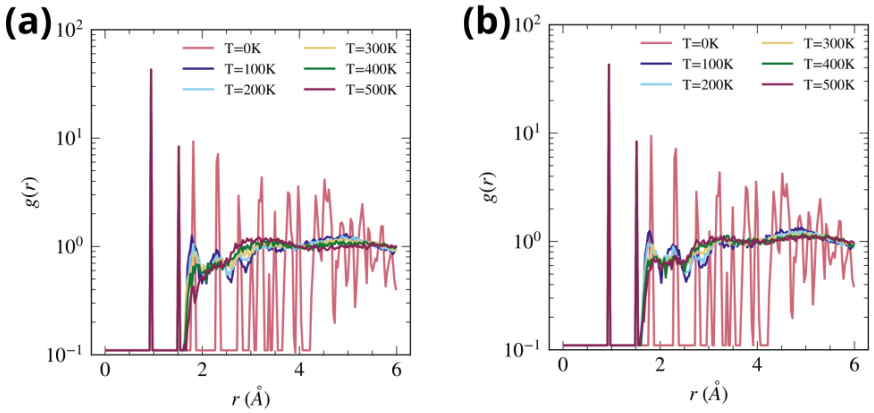

Figure 1.1: (a) The radial distribution function of (a) the TIP3P water model and (b) the TIP4P water model. The first peak at approximately 1 Å corresponds to the distance between the oxygen and hydrogen of the same water molecule, while the second peak at approximately 1.5 Å is due to the distance between the two hydrogens within the same water molecule.

The point at which the ice melts can be estimated by considering the temperature at which the number of O-H intermolecular bonds reaches a new equilibrium. Figure 1.2 shows that the melting temperature of the `TIP3P` is about 500 K while the `TIP4P` reaches equilibrium around 600 K. Both results differ significantly from the well-known value of around 300 K and reported simulation values of 146 K for TIP3P and 232 K for TIP4P [1] at pressures of 1 bar.

A possible reason for the large discrepancy between the simulated values of the melting temperature and the literature values in reference [1] is different simulated environments. Mainly, our simulations were integrated using a `nvt`, keeping the volume of the simulation cell constant, while allowing for differences in pressures. This is fundamentally different from every day experience and the provided simulations in which the pressure is maintained.

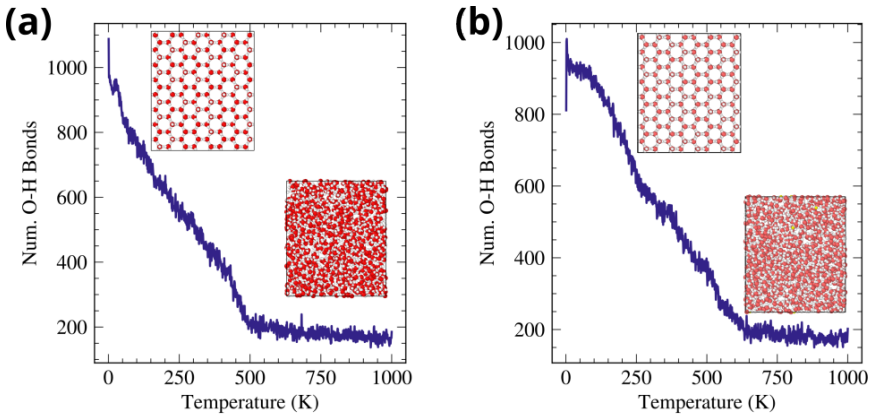

Figure 1.2: (a) **The number of OH bonds created using `Ovito` as a function of the simulation temperature for (a) the TIP3P water model and (b) the TIP4P water model. The insets show visualizations of the initial (left) and final (right) configuration of both simulations. The plateau of OH bonds indicates the melting of the ice showing a melting temperature of around 500 K for TIP3P model and 600 K for the TIP4P model.**

2 Assignment 2: Out of Flatland

2.1 Model Info

- *What force field is used?*

  The `Airebo-m.C` force field for carbon is used. It is a reactive force field, and more specifically an Adaptive Intermolecular Reactive Empirical Bond Order Potential.

- *Number of covalent bonds in the graphene sheet?*

  The graphene sheet contains 3840 carbon atoms, which makes sense as the graphene sheet was defined to contain 40x24 rectangular unit lattices of carbon, each containing 4 carbon atoms.

  The total number of covalent bonds in the graphene sheet is technically zero as no bonds were defined in LAMMPS, but rather interatomic interactions.

- *Key command line used to deform the graphene sheet?*

  The graphene sheet is deformed using the `fix indent` LAMMPS command as shown in listing 1. The command forms a sphere centered at the origin whose radius decreases at a fixed rate and indents all atoms within the sphere. To

```
variable r0 equal (ylo^2+xlo^2+zlo^2)^0.5
variable r0fix equal ceil(${r0})
variable rate equal 1.25      #A/ps
variable deltat equal "dt"
variable radius equal "v_r0fix-step*dt*v_rate"
fix constrain all indent 1 sphere 0 0 0 v_radius side in units box

# following command was used for tube/cylinder formation
fix constrain all indent 2 cylinder x -${yside} 0 v_radius side in units box
```

Listing 1: The key commands used for deforming the carbon sheet.

2.2 Running simulation

Figure 2.1 and 2.2 shows the midpoint and final configuration with the atoms colored by their potential energy for the spherical and cylindrical deformation, respectively.

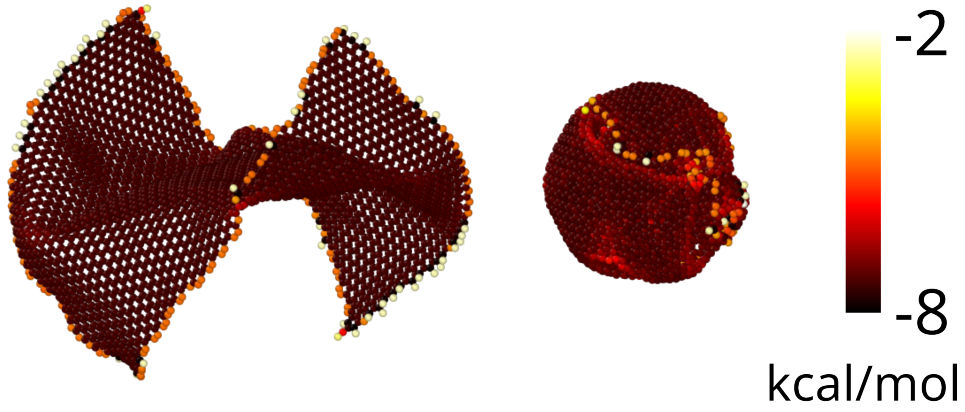

Figure 2.1: `Ovito` snapshots of the spherical deformation of the graphene sheet at the midpoint (left) and end (right) of the simulation. The coloring corresponds to the potential energy of each atom. Atoms at the edge have the highest potential energy.

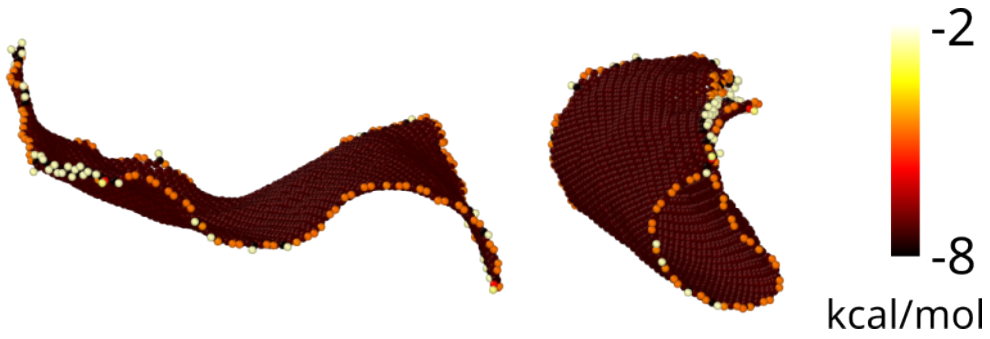

Figure 2.2: `Ovito` snapshots of the tube deformation of the graphene sheet at the midpoint (left) and end (right) of the simulations. The coloring corresponds to the potential energy of each atom. Atoms at the edge have the highest potential energy.

It is noticed that during the deformation, the atoms at the edge, in contact with the indent have the highest potential energy. Furthermore, it seems as if the spherical deformation leads to a more stressed system with a higher potential energy. This is confirmed when comparing the total potential energy further touched upon in Figures 2.3(a) and 2.4(a). Furthermore, the radius of gyration of the sphere is much smaller than that of the carbon nanotube (Figures 2.3(b) and 2.4(b)).

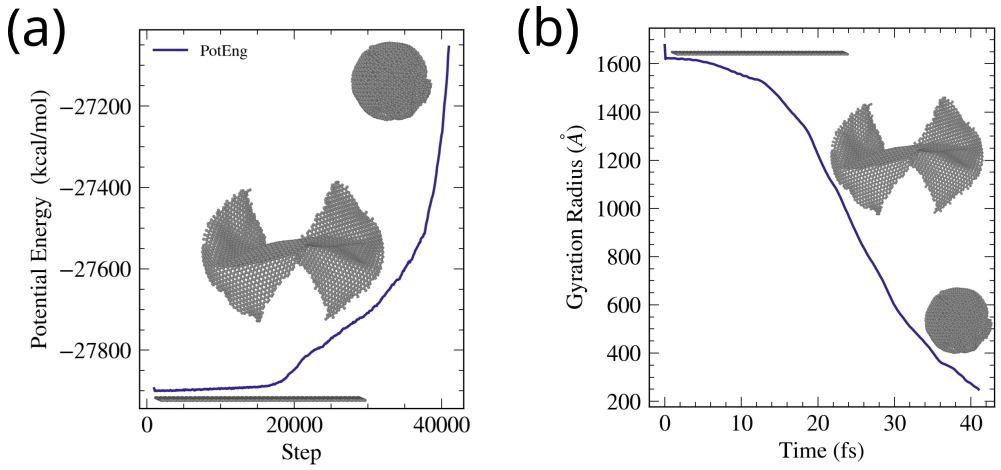

Figure 2.3: The (a) potential energy and (b) radius of gyration of the entire system throughout spherical deformation. The insets show (from left to right) the initial configuration, the midpoint configuration and the final configuration.

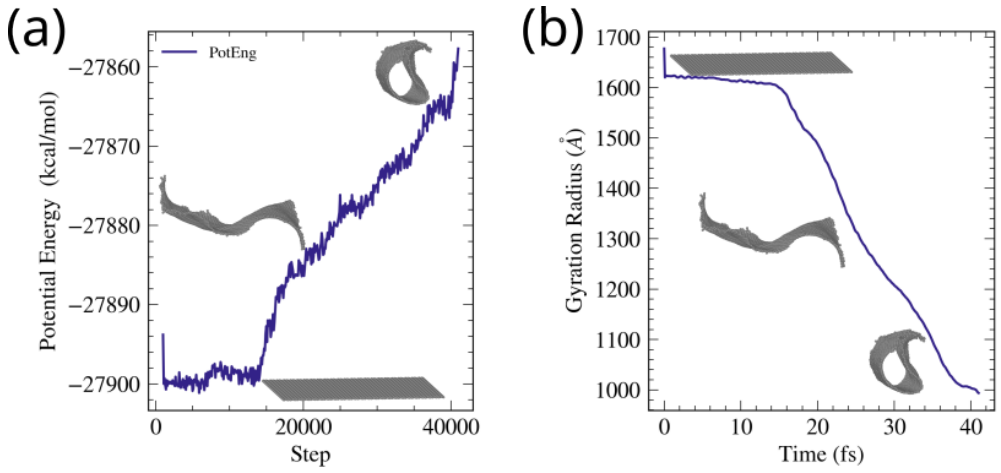

Figure 2.4: The (a) potential energy and (b) radius of gyration of the system throughout the cylindrical deformation. The insets show the initial configuration, the midpoint configuration and the final configuration.

2.3 Uniaxial deformation

Uniaxial deformation was achieved using the `fix deform` command with an engineering strain rate of 0.01 while the positions of the carbon atoms were remapped. Listing 2 shows the implementation of this.

```
# Creating uniaxial extention to graphene sheet
variable finalstrain equal 1.0
variable rate equal 0.01
variable deltat equal "dt"
variable numberofsteps equal round(${finalstrain}/(${rate}*${deltat}))

print "Deforming box in x direction from L(t) = L0 (1 + 0.01 * dt) for ${numberofsteps}
    steps"
fix extension all deform 1 x erate ${rate} remap x units box  # remapped set to positions
    not velocities
```

Listing 2: The key commands used for deforming the carbon sheet.

After a strain of 42.8%, the graphene sheet fails at multiple points as shown in Figure **??**. This is likely due to the fact that our simulated sheet is without impurities and thus initially without localized stress that can propagate.

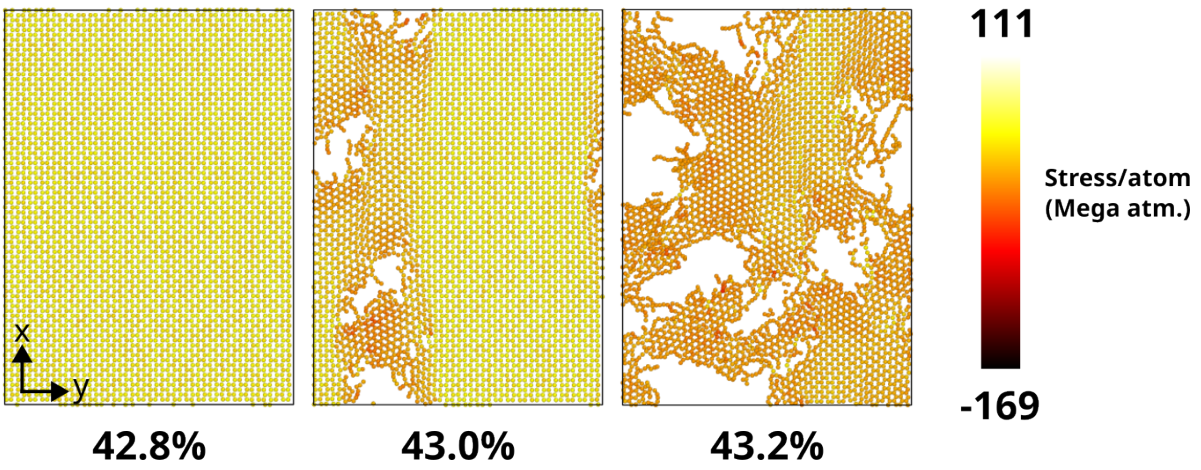

Figure 2.5: The progression of the carbon sheet failure with increasing system strain. The carbon atoms are colored by the stress/atom. The snapshots are taken within a timeframe of 0.6 fs. There is no specific point at which the failure propagates from.

3 Assignment 3: Go with the Flow

3.1 Model info

- *What force fields are used to simulate water and graphene?*

  The force fields used to simulate water and graphene are separated into bonded and non-bonded interactions. The bonded interactions include harmonic bonds, harmonic angles, opls dihedrals and harmonic impropers. The coefficients for these bonded interactions were set according to the GROMOS53A6 force field for carbon and the TIP4P force field for the water molecules.

  The non-bonded interactions were set using the `pair_style lj/cut/tip4p/long` command from LAMMPS. This sets the interaction between oxygen and hydrogen atoms to be that of the TIP4P force field, while other non-bonded interactions are a LJ potential.

```
pair_sytle lj/cut/tip4p/long 1 2 1 1 0.105 10
```

Listing 3: The command for the non-bonded interactions for water and carbon system. The first two numbers correspond to the atom ids to which the TIP4P force field applies. The next two numbers are the id for the O-H bond and the H-O-H bond angle, respectively. The second to last number is the distance of the massless charge site from the center of the oxygen. The final number is Coulombic interaction and global LJ cutoff.

- *What is the value of the interaction energy epsilon between oxygen and carbon atoms?*
- The interaction energy ϵ between the oxygen and carbon atoms is not directly specified, but determined by the arithmetic mixing rules:

$$\epsilon_{i \neq j} = \sqrt{\epsilon_i \epsilon_j} \tag{3.1}$$

  Which gives $\epsilon_{\text{O-C}} = \sqrt{0.18481 \cdot 0.0663} = 0.11069$.

- *Which command enforces the flow of water in the channel?*
- The `fix addforce` command enforces the flow of water in channel through the below implementation.

```
variable    frc equal 2e-4
group       water type 1 2 # 1 is O, 2 is H
fix         kick water addforce ${frc} 0.0 0.0
```

Listing 4: The command enforcing the flow of water in the channel. The flow is thus in the x-direction.

3.2 Running simulation

- *Calculate the layer-by-layer distribution of water molecules' mass and velocity profiles along the z-axis. How does the velocity profile compare to what is expected for macroscopic laminar flow? What is the reason for the discrepancy?*

  Both the velocity and mass profiles seem to be quite homogeneous throughout the channel, if we ignore the noise. This is quite different from the parabolic profile of laminar flow.

  The difference is likely due to the non-bonded interaction of the Oxygen and Carbon being set by the mixing rules, which can lead to results quite different than interactions optimized against the wetting angle of water on graphene [2].

  Furthermore, to achieve a laminar flow, there must be a no-slip boundary, which is not observed in our simulations.

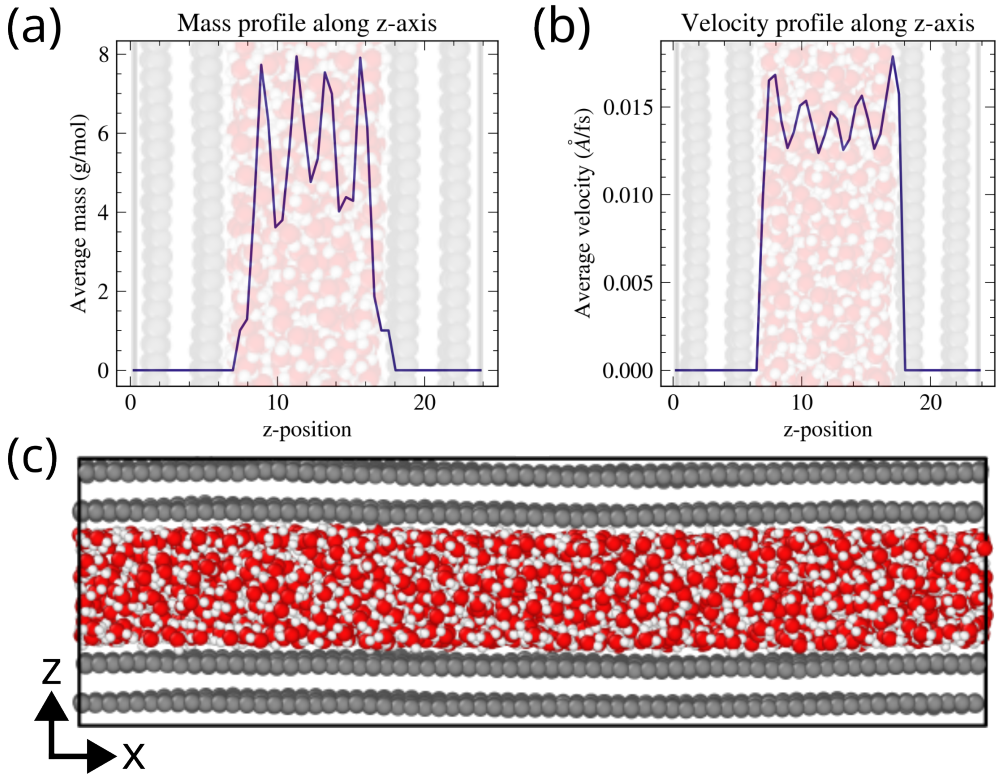

Figure 3.1: The results from running the simulations. (a) shows the layer-by-layer mass distribution while (b) shows the average velocity profile. (c) is an `Ovito` visualization of the channel.

- *To obtain Poiseuille flow, a bound layer of fluid is expected to be present near the tube walls. Where does this effect come from, and how would you address this in your simulation?*

  The bound layer present near the tube walls is due to the attractive interaction between the tube walls which cause the water atoms to stick on the wall. When the atoms diffuse away from the wall, their velocity increases due to momentum transfer between the diffusing water molecules and the flowing molecules. This is what leads to a parabolic velocity profile with a maximum at the center of the channel.

  To address the no-slip condition in the simulation, we can introduce regions within a certain distance from the carbon atoms forming the channel wall. Within this region, we can potentially enforce water molecules to stay stationary.

  To model the diffusion, the water molecules in the previously defined no-slip region could have a probability of being let free to move in line with a Boltzmann distribution.

References

(1) Vega, C.; Sanz, E.; Abascal, J. L. F. The Melting Temperature of the Most Common Models of Water. *The Journal of Chemical Physics* **2005**, *122*, 114507, DOI: 10.1063/1.1862245.

(2) Liao, S.; Ke, Q.; Wei, Y.; Li, L. Water-Graphene Non-Bonded Interaction Parameters: Development and Influence on Molecular Dynamics Simulations. *Applied Surface Science* **2022**, *603*, 154477, DOI: 10.1016/j.apsusc.2022.154477.

A Code for Velocity Profile

```
    # Set up binning along z-axis, values taken to match mass profile
    bin_count = 50
    z_min = 0.0
    z_max = u.dimensions[2]
    z_edges = np.linspace(z_min, z_max, bin_count + 1)
    z_centers = 0.5 * (z_edges[:-1] + z_edges[1:])

    # initializing arrays
    accumulated_velocity = np.zeros(bin_count)
    n_atoms = np.zeros(bin_count)
    average_vel = np.zeros(bin_count)

    # Compute speeds for atoms in water molecules for each trajectory
    for i, traj in enumerate(u.trajectory):
        waters = u.select_atoms("type 1 or type 2")
        positions = np.array(waters.positions.copy())
        velocities = np.array(waters.velocities.copy())

        # compute n_atoms and individual attom speed for each bin
        for j in range(1, len(z_edges)):
            mask = (positions[:,-1] >= z_edges[j - 1]) & (positions[:,-1] <= z_edges[j])
            n_atoms[j - 1] += len(positions[mask])
            for velocity in velocities[mask]:
                accumulated_velocity[j - 1] += np.linalg.norm(velocity)

    # Average accumulated speeds by total number of atoms
    for idx in np.nonzero(n_atoms):
        average_vel[idx] = accumulated_velocity[idx] / n_atoms[idx]
    plt.plot(z_centers, average_vel)
```

Listing 5: Main code snippet for determining the velocity profile in the channel.

```
def compute_linear_density(atomgroup, trajectory_slice, axis = 'y', binsize = 1.0):
  """
  Compute the linear mass density of an atom group over a trajectory.

  Parameters:
      atomgroup: MDAnalysis AtomGroup
          The group of atoms to analyze.
      trajectory_slice: iterable
          A slice or iterable of trajectory frames.
      axis: str
          Axis along which to calculate density ('x', 'y', or 'z').
      binsize: float
          Bin width for the density calculation.

  Returns: np.ndarray
      2D array of linear mass density per frame with the shape (num_frames, num_bins)
  """

  num_frames   = len(trajectory_slice)
  max_box_size = max(atomgroup.universe.dimensions[:3])
  # num of columns is always equal to largest_dimension/binsize
  num_bins = int(max_box_size / binsize)

  density_analyzer = lineardensity.LinearDensity(atomgroup, binsize=binsize)
  density_array = np.zeros((num_frames, num_bins))

  for ts in trajectory_slice:

      frame_idx = ts.frame
      density_analyzer.run(frames=[frame_idx], verbose=False)
```

```
        density_array[frame_idx,:] = density_analyzer.results[axis]['mass_density']

    return density_array
```

Listing 6: The locally modified version of the linear `compute_linear_density` function of `beadspring` package.